

Programmable character generator for your S100 system

Craig Barratt

Add creativity to your computer! Another super project in our S100 bus computer series — this PCG teams with the popular ETI-640 VDU.

NOW THAT you have had your ETI-640 VDU for a while and can write characters anywhere on your monitor, wouldn't it be nice to be able to create your own characters and use these at will?

Well, we thought that this would be a great idea and after saying the right incantations a programmable character generator appeared.

It was soon discovered that the board also contained a logic probe for debugging and fully implemented joysticks for that once dreamed of analog input. And the people were happy. Well, you may ask, who would want a programmable character generator? I could just as easily ask who would want a microcomputer in their bedroom!

The PCG is in fact a very useful device when used in conjunction with the ETI-640 VDU. The most obvious use is to simply write a new character set into the PCG and use the VDU as you would normally. Instead of the usual characters appearing on the screen, the characters which you created would appear instead. This concept may be extended simply to creating special character sets for programming languages (APL requires a special character set), or having realistic aliens and starbases in your next game of Startrek. You might even play a chess game with the screen really looking like a chess board with recognisable pieces on it.

Besides passively generating character sets for use by programs, you can have programs change characters as you want. This means shapes can be moved across the screen and very realistic games can be produced.

So a PCG will let you win Wimbledon and destroy the enemy tanks — all from your own bedroom!

ETI 681

Volume

= area x height

= area x h

= $\pi r^2 \times h$

= $\pi r^2 h$.

Surface area

= $2 \times \pi r^2 + 2\pi r \times h$

= $2\pi r(r + h)$.



$\leftarrow r \rightarrow$

The ETI 640 font

0 1 2 3 4 5 6 7 8 9 : ; < = > ?

@ A B C D E F G H I J K L M N O

P Q R S T U V W X Y Z [\] ^ _

` a b c d e f g h i j k l m n o

p q r s t u v w x y z { | } ~

Did you know that:

$$\phi = \phi^2 - 1 = \phi^{-1} + 1 = \frac{1}{2}(\sqrt{5} + 1) \approx 1.618033988...$$

An example of the range of 'tricks' possible with this programmable character generator

'Just one more toy' I hear all the purists say. Well, there is something in it for you purists too. With some good driving software you can use the PCG for x,y graphics with the joysticks as analog input, but more about this later.

Features

Apart from being able to program up to 128 characters with the PCG, you can still use all the 128 characters available from the ETI-640. For each of the 1024 characters on the screen you can select whether it is a PCG or a ROM character. Of course, inverse and flashing characters (PCG or ROM) as well as chunky graphics, are still available on the ETI-640 VDU.

As shown in the accompanying diagram each of the 1024 characters on the ETI-640 screen consists of a matrix of bits — 16 bits high and eight bits across. So that characters are not pushed together, gaps are left around each one. Thus, a normal character from the ETI-640 actually sits in a smaller matrix of 12 bits high and seven bits across. The space around this smaller matrix appears black on the screen and is the gap separating characters.

When chunky graphics appear on the screen they completely occupy the 16 x 8 matrix and thus they can run together allowing unbroken lines and pictures to be drawn.

A character created by the PCG gives ►

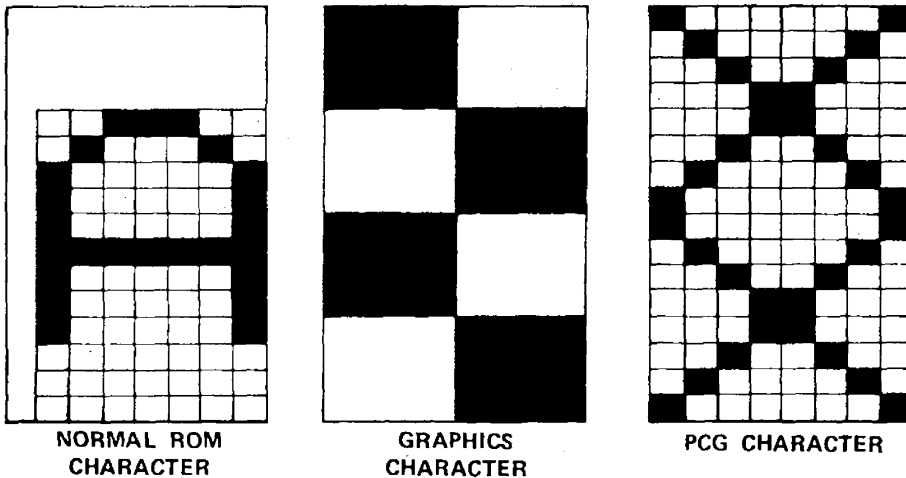


Figure 1. The character and graphics matrices for the ETI-640 VDU compared to the 16 x 8 matrix of the ETI-681 programmable character generator.

you the best of both worlds. With the PCG you can program every bit in the 16 x 8 matrix. Thus you can get the same definition as with normal characters but PCG characters can run together, like graphics characters.

The PCG occupies 2K of processor memory, which is located using an on-board DIP switch. This amount of memory is required since each character requires 16 bytes to define and 128 characters can be programmed. The representation of a PCG character is shown in Figure 2.

LOW NIBBLE	BIT NUMBER
0	7 6 5 4 3 2 1 0
1	7 6 5 4 3 2 1 0
2	7 6 5 4 3 2 1 0
3	7 6 5 4 3 2 1 0
4	7 6 5 4 3 2 1 0
5	7 6 5 4 3 2 1 0
6	7 6 5 4 3 2 1 0
7	7 6 5 4 3 2 1 0
A	7 6 5 4 3 2 1 0
B	7 6 5 4 3 2 1 0
C	7 6 5 4 3 2 1 0
D	7 6 5 4 3 2 1 0
E	7 6 5 4 3 2 1 0
F	7 6 5 4 3 2 1 0

Each row of a character is one byte of memory. The most significant bit (bit 7) is the left hand bit as it appears on the screen, right through to the least significant bit (bit 0) which is the right hand bit as it appears on the screen. The top row of a character has a low nibble address of 0, the second has an address of XXX1 (hex).

Since each character occupies 16 bytes of memory, for a PCG starting at

hex 'F800', character '00' in the PCG may be found at location hex 'F800'. Similarly, character '01' in the PCG may be found at location hex 'F810', character '02' at location hex 'F820' and so on. To convert between a character number and a PCG address, simply put a zero onto the end of the character number (in hex) and add the new number to the base address of the PCG. For example, character hex '5B' will be found at location hex 'FDB0' in the PCG, since hex 'FDB0' equals hex 'F800' plus hex '5B0'.

If you recall, there are two bits in the function block of the ETI-640, the flashing bit and the graphics bit. Using these bits you can select whether each character on the screen is flashing or graphics. The PCG adds a third bit to the function block of the ETI-640. Setting this bit low will make that character a normal (ROM) character. Setting it high will make that character a PCG character. The graphics and flashing bits operate as usual.

The Joysticks

Up to four joysticks may be connected to the PCG. The joystick circuitry is totally independent of the PCG's character generation. The joysticks are I/O oriented while the PCG is memory-mapped.

A joystick consists of two potentiometers, one varying with vertical movement of the joystick paddle and the other with horizontal. Thus, up to eight pots may be connected to the PCG. The lines that interface to the pots are known as channels and eight of these inputs are available on a DIP socket on the PCG board.

The processor controls the joysticks through one I/O port. To get the value of

a given input channel the processor writes out the channel number to the joystick port. This initiates the timing cycle and the top bit of the input port goes low when the conversion is complete. The remaining seven bits of the input port are the converted value. The joystick circuitry in fact has other goodies built in, but more about this later.

The logic probe

A logic probe has been included on the board to aid in the service of this card and others. Two LEDs, red and green, indicate high and low levels respectively. If the logic probe is connected to a tri-state signal, the LEDs will flash slowly. This means the probe will give responses in the following way:

Condition of input	Logic probe output
High level	Red LED lit, green LED out.
Low level	Green LED lit, red LED out.
Tri-state level	Green and red LEDs alternately flashing.
Pulse train	Green and red LEDs both partially lit.

When looking at a pulse train, the brightness of each LED approximately corresponds to the time for which the pulse train is in that state. For example, a bright red LED and a slightly lit green LED means the input is mostly high and pulses low for short periods of time.

Construction

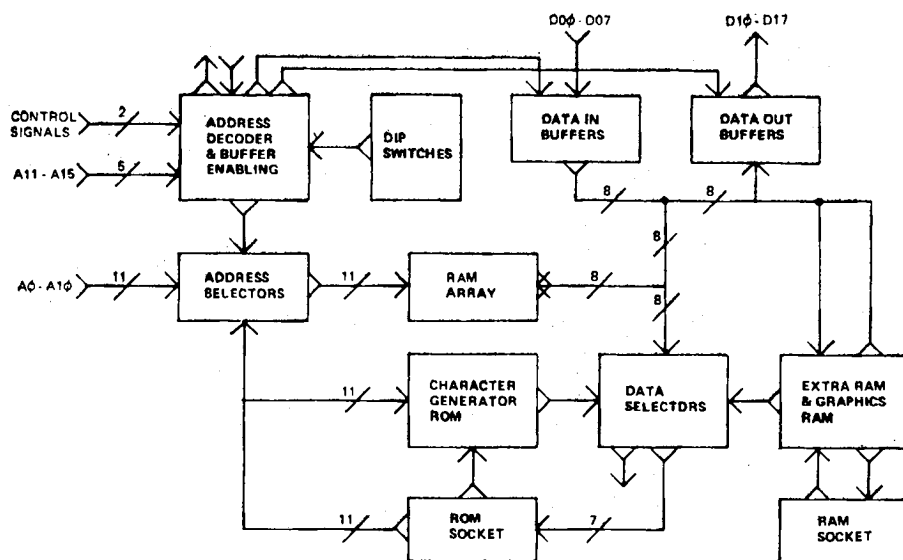
Now that you are convinced that you "just gotta get one", the next step is to get the parts and build up a board. Before you rush out and buy the parts, do yourself a favour and give this section a good read.

A good place to start would be the printed circuit card. Due to the large number of holes that need plating through, and the number of tracks that connect to chip pins on the top side of the board, using a plated-through hole board is strongly recommended. Such boards will be made available through Applied Technology we understand. For those incorrigible diehards who want to do it the hard way the PCG pc board patterns are not printed in this magazine. Don't despair! Send a large stamped, self-addressed envelope to 'PCG PCBs', ETI Magazine, 15 Boundary Street, Rushcutters Bay, NSW 2011 and we will send the patterns back.

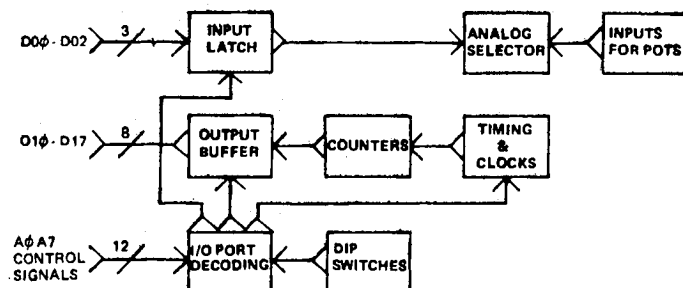
If you are making your own printed circuit board you should start by soldering links through the board in positions which will be later covered by ICs.

The board has been carefully designed so that as few tracks as possible pass between IC pins on the

prog. character generator



BLOCK DIAGRAM OF THE PROGRAMMABLE CHARACTER GENERATOR



BLOCK DIAGRAM OF THE PCG JOYSTICKS

solder side. Despite this, you should take care with each joint, watching out for dry joints and solder splashes.

There are two chips that you don't have to buy when you build the PCG. These are ICs 7 and 13 which are a 2102 RAM and the character ROM respectively. These come from the ETI-640, where they live under the alias of IC 22 and IC 4 respectively. DIP-to-DIP leads connect DL1 to the ex-IC22 and DL2 to the ex-IC4 sockets on the ETI-640, but more about this later.

Construction should commence with the insertion and soldering of all IC sockets, if you are using them. If you are not using sockets for the ICs you should still have sockets for the DIP-to-DIP leads, DL1, DL2 and DL3. Use a three-pin Utilux socket, ETI-632 VDU style, for CN1. Also solder in the DIP switches, DS1 and DS2.

All the resistors should be inserted next, followed by all the capacitors, except C21 whose value depends upon the value of the joysticks used. Note the polarity of C1, C2, C3 and C21 (when

inserting them). The diodes and transistor are soldered in next, again noting polarity, followed by the potentiometers RV1 and RV2 and finally the heatsink with IC34.

Note that RV1 and RV2 are multiturn trimmers. These are used to set up the joysticks and it was decided that multiturn trimmers would make setting them up a lot easier.

Choosing components:

Some of the components used will vary from PCG to PCG (if more than one person builds one). These variations are:

- 1) If you are connecting the PCG to a processor board which has an inverted data bus (such as the KT9500), ICs 2, 19, 20, 21, 22 and 27 will have to be 74LS368s instead of the 74LS367s as specified in the parts list.
- 2) The value of C21 depends on the value of the joysticks used. Choose an appropriate value from Table 1.

If the value of the joystick that you are using is not in Table 1 then simply apply the magical formula provided.

VALUE OF RV1/RV2	VALUE OF C21
5k	330n
50k	33n
100k	18n
MAGIC FORMULA	
$C21 = \frac{1.75}{R_p + 0.3}$	
C21 is the value of C21 in μF R_p is value of the pots in the joysticks; in kohms.	
TABLE 1	

Before inserting any ICs it is a good idea to apply power to the board and check the output of the 5V regulator. When this voltage is correct, solder or plug in the ICs and, besides interfacing the PCG to the ETI-640, construction is complete.

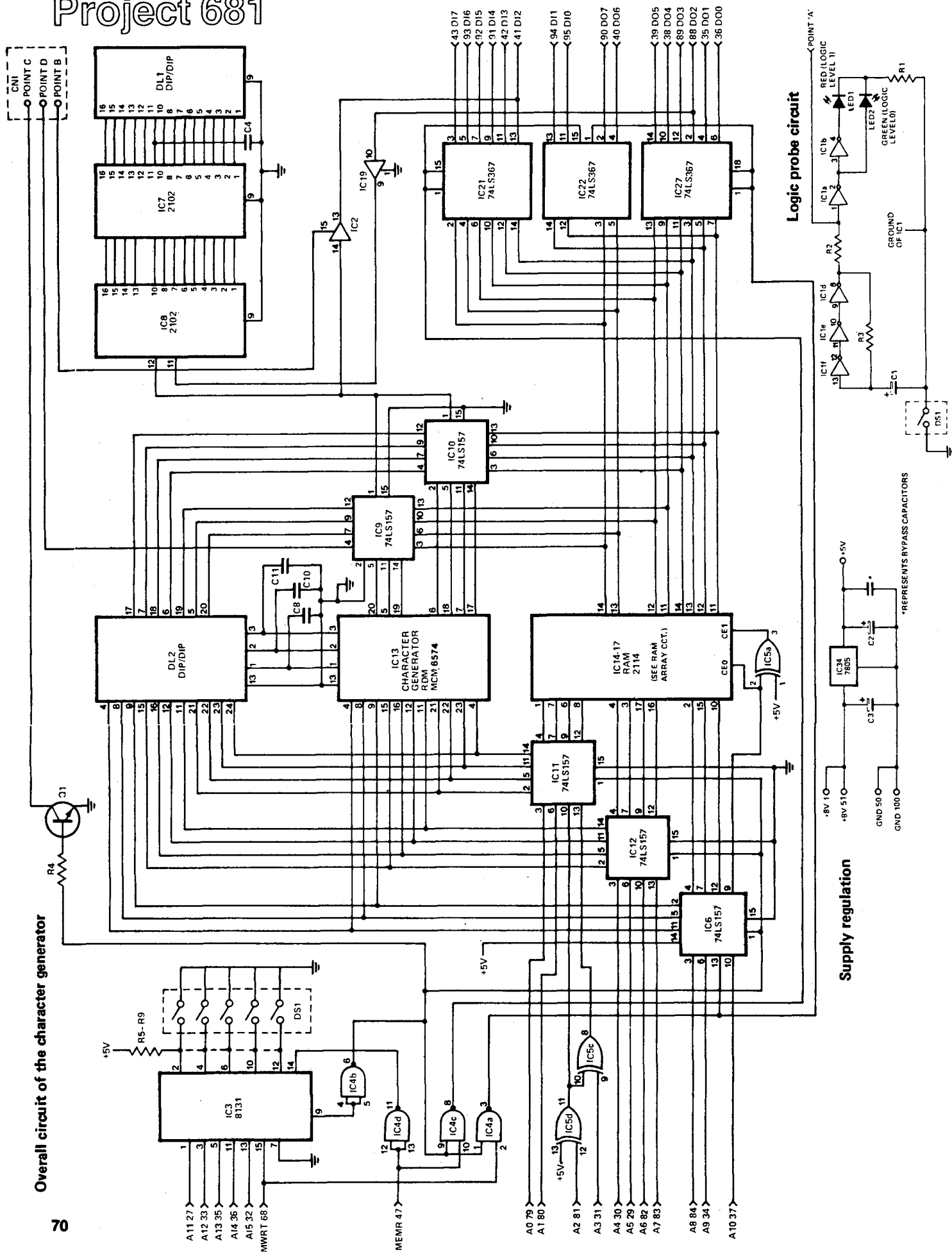
Before interfacing the PCG to the ETI-640 you might like to power the board up, with IC13 (the character generator ROM) still in the ETI-640, where it is known as IC4. Make sure that no large blue sparks, smoke or bad smells appear when you power up the board. Run the backs of your fingers over the ICs for a few seconds to ensure that nothing is getting too hot.

If you have an S100 bus system, simply plug in the PCG conveniently near the ETI-640. In this case it is assumed that all signals that are required to drive the PCG are on your bus. The signals that the PCG requires are very simple and you shouldn't exhibit any problems with the idiosyncrasies of the S100 bus.

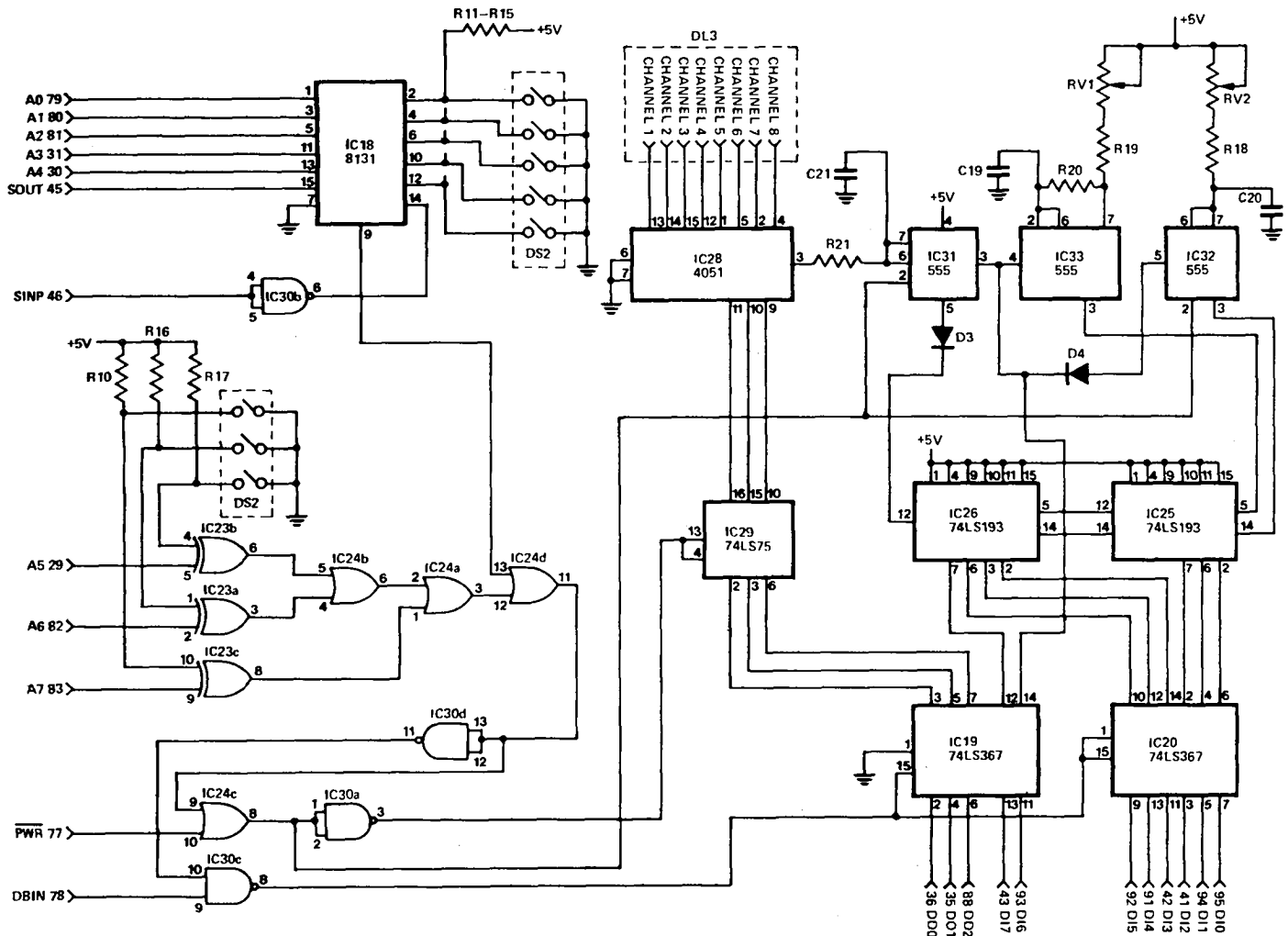
If you don't have an S100 bus system then you are in for another wrap/solder operation. The "spare socket" as marked on the overlay may be used for extra gates to derive some sort of S100 signals for the PCG. Since you already have an ETI-640 on your system you already have MWRT and MEMR signals. All you will have to generate is OUT and INP. Refer to the ETI September 1977 issue for more details on these signals.

Project 681

Overall circuit of the character generator



prog. character generator



Overall circuit of the joystick section

HOW IT WORKS — ETI 641

The functional description of the PCG board may be conveniently divided into three parts: the logic probe, the PCG, and the joysticks.

THE LOGIC PROBE:

IC1a and 1b drive the green (logic 0) and red (logic 1) LEDs respectively. Since both aren't on together only one dropping resistor, R1, is required. When the input, point 'A', is low IC1, pin 2 will go high, forcing the green LED on.

IC1d, 1e and 1f are wired as an oscillator running at a frequency of about 5 Hz. When point 'A' is open circuit, or connected to a tri-stated signal, the LEDs will flash at this rate, indicating a tri-state condition. Since point 'A' is being driven by this oscillator through R2, some signal distortion may occur, particularly when looking at open collector signals. If this is a problem R2 should be removed and tri-stated signals will appear as logic 1 (red LED).

PCG:

In normal running mode (no processor accesses), buffer ICs 21, 22 and 27 are turned off and selector ICs 11, 12 and 6 are driving RAM addresses with character and row information from the ETI-640 VDU via DIP lead DL2. R/W is forced high (read) and C5a inverts the most significant address so that either the lower 1K (ICs 15 and 17) or the upper 1K (ICs 14 and 16) is enabled.

Appropriate dot data is produced by both the RAM (ICs 14 to 17) and the ROM (IC13). IC9 and IC10 select whether data from the ROM or RAM is supplied back to the ETI-640. These selectors are switched by the state of the ROM/PCG RAM bit (IC8). The addresses for IC8 come from the ETI-640 via DIP lead DL1. The displaced 2102 chip becomes IC7 and still functions normally as the graphics/normal bit.

Input data (DO2) for IC8 is provided by IC19. R/W for IC8 is provided by the ETI-640. Output data is gated onto the bus by IC2 whenever a read from ETI-640 memory is made. Since this read condition cannot simply be detected by the PCG it is taken from the ETI-640 via point 'B'. The eighth bit from RAM is taken back to the ETI-640 via point 'D' because only seven bits are available on DL2.

IC3 compares A11 to A15 with the DS1 settings. IC3 pin 9 will go low when the processor is doing a read or write to the PCG memory. IC4a and IC4c turn on the RAM data input buffers (ICs 22 and 27) and the RAM data output buffers (ICs 21 and 22) respectively. During processor accesses incorrect character information will be given to the ETI-640. To prevent these 'flickers', Q1, via point 'C', blanks the ETI-640 screen.

Selector ICs 6, 11 and 12 supply processor addresses to the RAMs during accesses. IC5c and IC5d add 12 (decimal) to the processor

address (without carry above A2) because the top row of each character has a row address of 12 (decimal) on the ETI-640. Refer to ETI April 1978 page 35 for an explanation.

JOYSTICKS:

The joystick conversion sequence is triggered by a write to the joystick port. ICs 18, 23 and 24 detect a port access to the port address selected by DS1.

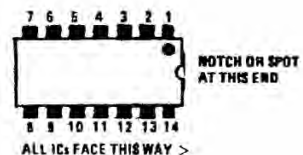
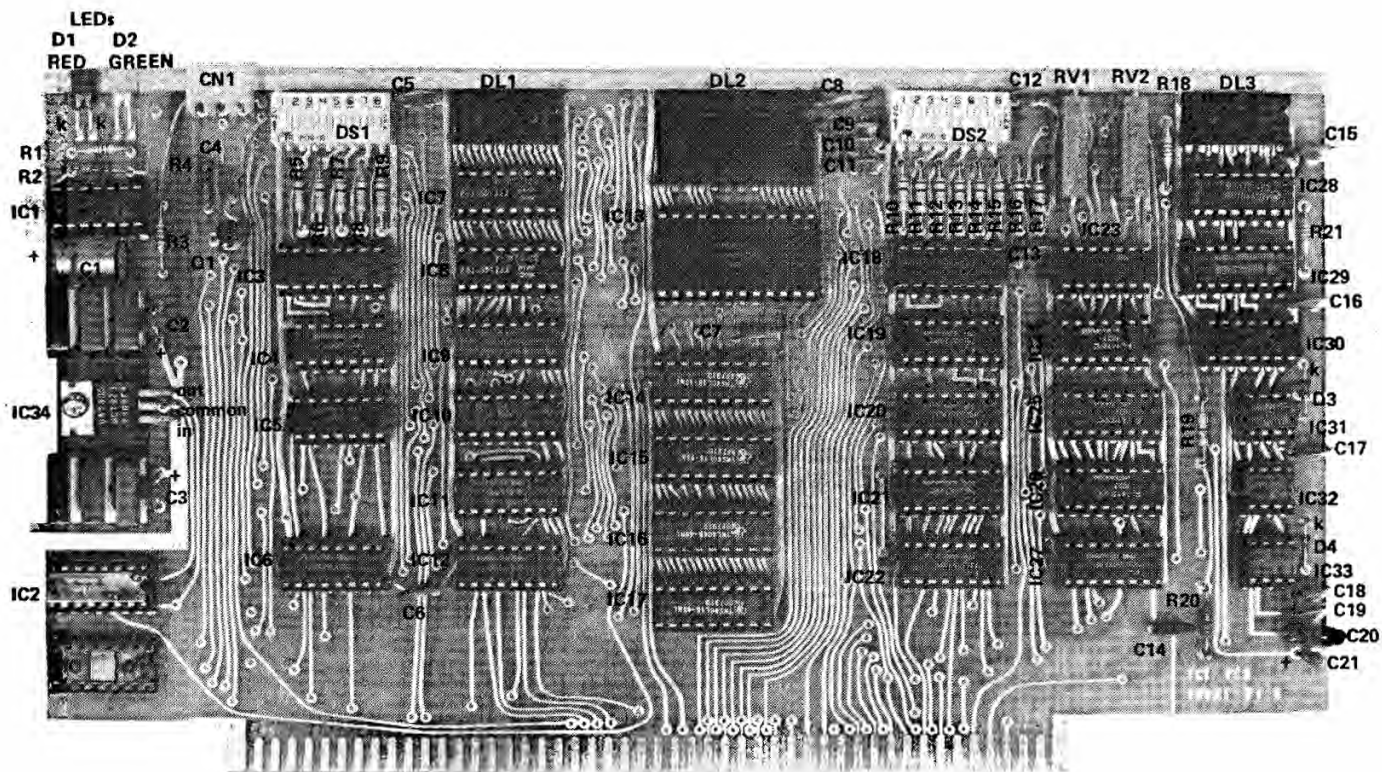
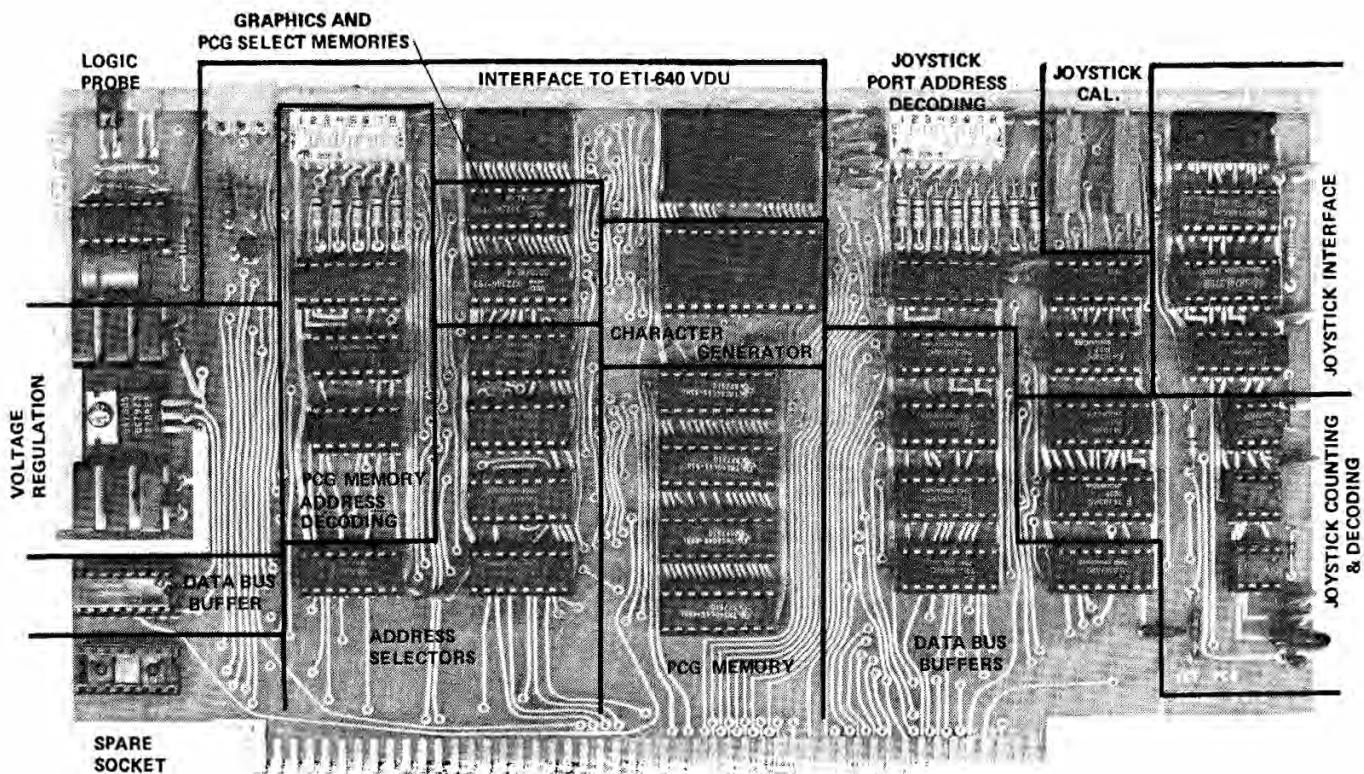
A write to the joystick port latches the lower three bits of the data bus (IC29). This is the channel number. The write also triggers ICs 31 and 32, both monostables. IC32 holds the counter ICs 25 and 26 cleared for a period set by RV2. The period of IC31 is proportional to the resistance selected by IC28.

After being held cleared for the duration of IC32, counter ICs 25 and 26 count the pulses from IC33 for the duration of IC31. If the count reaches 127 (decimal), the maximum value, the monostable is reset prematurely by D3.

The output of IC31 is the top bit of the joystick input port. When this is low the remaining seven bits represent the converted value of the input channel.

IC30c detects a read from the joystick port and turns on ICs 19 and 20 so that the outputs of ICs 25, 26 and 31 may be put onto the processor's input bus.

Project 681



prog. character generator

PARTS LIST - ETI 681

Resistors all 1/4W, 5% unless noted

R1, R18-19, R21 .330R
R2, R4, R20 . . . 1k
R3 1k9, 2%
R5 - R17 10k
RV1, RV2 5k multiturn trimpot

Capacitors

C1, C2 10u tantalum, 6V
C3 100u electro
C4 - C18 22n ceramic
C19 1n ceramic
C20 82n ceramic
C21 see text

Semiconductors

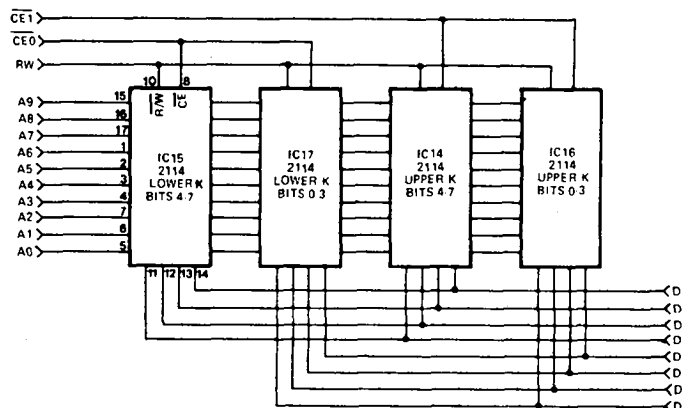
D1 red LED, TIL220 or sim.
D2 green LED, TIL222 or sim.
D3, D4 1N914
Q1 BC548
IC1 74LS514
IC2, IC27, and
IC19-22 74LS367
IC3, IC18 8131
IC4, IC30 74LS00
IC5, IC23 74LS86
IC6, IC9-12 . . . 74LS157
IC7, IC8 2102, see text
IC13 MCM6574 or MCM6674, see text
IC14-IC17 . . . 2114
IC24 74LS32
IC25-IC26 . . . 74LS193
IC28 4051
IC29 74LS75
IC30 74LS00
IC31-IC33 . . . 555
IC34 7805 or LM340T-5

Miscellaneous

pc board ETI-681
DS1, DS2 8-way DIP switches
DL1 16-pin DIP-to-DIP lead
DL2 24-pin DIP-to-DIP lead with one inverted end
DL3 16-pin DIP-to-bare wire lead
CN1 3-pin pc mount plug-socket set

Two 24-pin IC sockets; four 18-pin IC sockets; 22 16-pin IC sockets; six 14-pin IC sockets; three 8-pin IC sockets; "flatpack" heatsink and mounting/insulating hardware for IC34.

NOTES: If you have an ETI-640 VDU board then IC7 and IC13 above will come from the ETI-640 board. The DIP switches are not socketed. Resistors R5 through R17 may vary one standard value either side of that given as these components are not critical — the same goes for C1, C2 and C4 through C18. All TTL ICs may be either normal TTL chips or TTL LS types, any mix of these types should function normally in this project. The value of C21 depends on the value of the joystick pots and the method of obtaining its value is given in the text.



The RAM array circuit — see also complete circuit on page 70.

Interfacing

The next major step is to interface the PCG to the ETI-640. This is done in three steps:

- Firstly, unplug the character generator MCM6574 or MCM6674 and the 2102 graphics select RAM from the ETI-640 and plug them into the PCG. Their ETI-640 chip numbers are IC4 and IC22, and their new PCG numbers are IC13 and IC7 respectively.
- Two DIP leads connect the ETI-640 to the PCG. One 16-pin DIP-to-DIP lead connects DL1 on the PCG to the position previously occupied by IC22 and one 24-pin DIP-to-DIP lead connects DL2 on the PCG to IC4 on the ETI-640. Note that the 24-pin DIP cable has pins pointing in one direction at one end and the other way at the other end. This isn't essential, but avoids the need to twist the cable, which could look messy. With the right angle bend in the 16-pin DIP lead and the strange or twisted 24-pin DIP lead, make extra sure that their orientations are correct, i.e: make sure that pin 1 of IC22 is connected to pin 1 of DL1 and not pin 9!
- Finally, three connections must be made through connector CN1. These wires connect points "B", "C" and "D" on the PCG to various points on the

ETI-640. Connect these lines as follows:

- Connect a wire between pad "B" and pin 8 of IC33 on the ETI-640.
- Connect a wire between pad "C" and pin 12 of IC16 on the ETI-640.
- i) Cut the pad between pins 14 and 15 of IC5 on the ETI-640, ensuring that the track going to these pads is still connected to pin 15.
ii) Connect a wire between pad "D" and pin 14 of IC5 on the ETI-640.

The PCG is now (hopefully) correctly interfaced to the ETI-640.

Setting up

Having completed all the work associated with "construction" and "interface", there is more to come! (Sorry to disappoint you.) The next thing to do could be broadly defined as "setting up" the PCG. This involves setting the DIP switches, setting up the joystick circuitry and testing the PCG.

DIP switch No. 1:

DIP switch 1, or DS1 has two functions: these are to locate the PCG's 2K of memory at any 2K boundary in your microcomputer's address space and to turn the logic probe on and off.

DS1 is functionally illustrated in Figure 2. Locating PCG memory using ▶

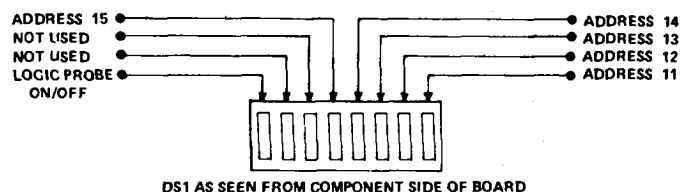
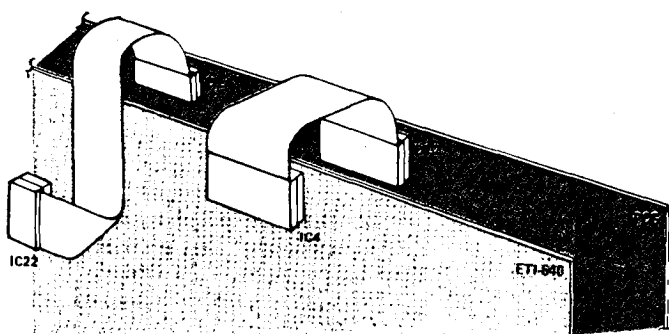


Figure 2. Functional arrangement of the DIP switch DS1.

LEFT: Linking the ETI-681 to the ETI-640 VDU.



Project 681

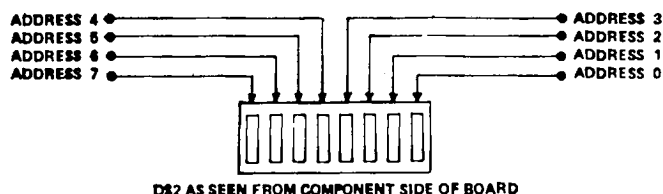


Figure 3. Functional arrangement of the DIP switch DS2.

DS1 is much the same as locating the ETI-640's memory. To locate the board at a certain address simply set that address on the DIP switch, remembering that a closed switch represents a "0" for that address. For example, to locate the PCG at hex 'E800' set the address switches to off, off, off, on and off.

For software compatibility we recommend that you put your PCG at 'F800' if your ETI-640 is at 'F000', or at '7000' if your ETI-640 is at '7800'. If your ETI-640 is at neither of these locations then choose any convenient address and enjoy converting software so that it will run in your system! The operation of the on/off switch for the logic probe is obvious.

DIP switch No. 2:

This DIP switch is used to locate the I/O port that the joysticks use. It is functionally shown in Figure 3 and operates in a similar fashion to DS1.

DS2 can locate the joystick port to one of 256 possible I/O ports (0 to 'FF' in hex). We recommend that you put the joystick port at hex 'FF', for no particular reason other than that we don't know of anyone's I/O that will conflict with this port.

If too many people are already using this port, strongly object and we will try to move it.

Testing the PCG:

Well, now comes that big moment. Although you know that nothing explodes when you power up, you don't know whether it works. There is only one way to find out — plug your system back together and power up. You should be confronted with a screen absolutely full of garbage!

You should have flashing, graphics, normal and random PCG characters all appearing.

Firstly, clear the screen using your system monitor and set the top line of the screen to non-scrolling if you have a scrolling monitor. Then do the following:

- 1) At 'F800' store hex 80, 40, 20, 10, 08, 04, 02, 01, 01, 02, 04, 08, 10, 20, 40, 80.

- 2) At 'F000' store '0'.
- 3) At 'F400' store '04'.

In the top left hand corner of your screen you should see an extended "greater than" sign.

The joysticks:

Up to eight pots may be connected to the joystick circuit, each connected to an input channel. These channels are found on DL3, and the layout of DL3 is shown in Figure 4.

To connect a pot to a given channel simply connect the pot wiper to one end of the carbon track and also to +5 volts. Connect the other end of the pot to a channel input.

While we could simply tell you how to set up the joysticks we feel that it would be better if you had some understanding of what you are really setting up.

As the resistance of the joystick varies, the converted value will vary between 0 (minimum resistance) and hex '7F' (maximum resistance). Figure 5 is an attempt to show what the joystick circuit produces as a converted value. Period 1 is the time for which the count stays at a value of 0 and is typically 0.1ms. Period 2 is the time for which the count increments to a value of hex '7F'. Period 3 is where the count stays at hex '7F'. This whole counting sequence is stopped after Period 4 which is proportional to the resistance of the selected pot. Thus, as the joystick is moved from minimum resistance there will be a band for which the count will be 0 (set by RV2), a band in which the count will count up to 127 (hex '7F'), set by RV1, and a band for which the count will be 127 as the joystick is moved to maximum resistance.

The method for setting up the joysticks is outlined below, and simply involves setting RV1 and RV2.

- 1) Firstly connect a pot, or half a joystick, to channel 1 on DL3.
- 2) Write a program to
 - a) write a '0' to port 'EF' hex (or your joystick port).
 - b) read port 'EF' until bit 7 (MSB) is low.
 - c) display the remaining 7 bits on the ETI-640 in hex.

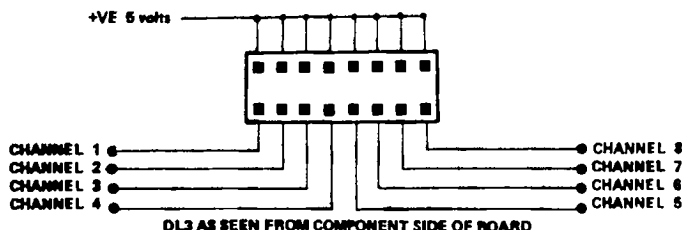


Figure 4. How the joystick pots connect to DL3.

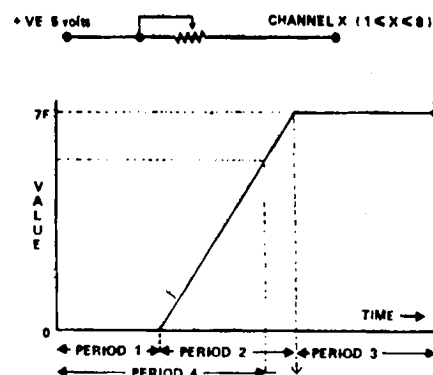


Figure 5. Illustrating the relationship between the joystick 'value' and the counting/timing sequences.

- d) jump back to a).
- 3) Run the program.
- 4) Set the pot to a position for which you want band 1 to last.
- 5) Set RV2 until the reading on the screen is just 00.
- 6) Set the pot to a position for which you want band 2 to last.
- 7) Set RV1 until the reading is just '7F' hex.

The setting of the joystick circuit is now complete. Stretch band 1 and compress band 2 a little if you cannot get the full count range (0 to 127) for each joystick that you connect.

If you try to read the value of an unconnected channel the conversion sequence will never be completed. If your processor locks up while using the joysticks this is a possible reason. While we were designing the PCG we decided not to do anything about this illegal condition since it shouldn't occur anyhow, and if it does it is a good indication that something is amiss.

When Dave Griffiths designed the ETI-640 he made life easier for himself making the VDU clock out nine bits for each character. The ninth bit takes on the same value as the eighth bit.

In fact, we didn't even notice this ourselves; it wasn't until someone fired up the PCG with a good monitor and began drawing diagonal lines on the screen that the problem was noticed! You shouldn't be unduly concerned with this problem; in fact, we doubt that many of you would have noticed it if we didn't mention it!